

DD: WO-01-CONTRACT-FOR-SUB-WF — How subscription workflows call the Work Order module

Developer implementation guide: DD_WO-01-CONTRACT-FOR-SUB-WF-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_WO-01-CONTRACT-FOR-SUB-WF-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management × Group III: Fulfillment | **Version:** v1.0 | **Companion to:** DD_WO-01-FRAMEWORK-v1.0, DD_WO-01-FLOW-SHIFTING-v1.0 Contract for subscription-workflow DDs (SUB-WF-ACTIVATE-01, SUB-WF-RELOCATION-01, SUB-WF-MIGRATION-01, SUB-WF-TERMINATE-01) that create or await Work Orders. Defines which WO kinds + endpoints + Kafka events + correlation fields to use. The WO module itself is specified by DD_WO-01-FRAMEWORK and per-kind FLOW satellites; this DD describes the consumer-side integration surface only.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/07_work_order/DD_WO-01-CONTRACT-FOR-SUB-WF-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- POST /api/work-orders
- POST /api/work-orders/{id}/cancel

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- ful-call-claim-homepass
- ful-call-release-homepass
- osr-equipment-bind
- osr-equipment-unbind

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionTerminated
- WorkOrderCancelled
- WorkOrderCreated
- WorkOrderFinalized
- WorkOrderPhaseFinalized
- WorkOrderPhaseTransitioned
- WorkOrderTimedOut

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management × Group III: Fulfillment | **Version:** v1.0 | **Companion to:** DD_WO-01-FRAMEWORK-v1.0, DD_WO-01-FLOW-SHIFTING-v1.0

Contract for subscription-workflow DDs (SUB-WF-ACTIVATE-01, SUB-WF-RELOCATION-01, SUB-WF-MIGRATION-01, SUB-WF-TERMINATE-01) that create or await Work Orders. Defines which WO kinds + endpoints + Kafka events + correlation fields to use. The WO module itself is specified by DD_WO-01-FRAMEWORK and per-kind FLOW satellites; this DD describes the consumer-side integration surface only.

Glossary

Term	Meaning
WO kind	High-level discriminator (INSTALLATION / SUPPORT / SHIFTING / TECHNOLOGY_MIGRATION / operator extensions). Drives BPMN selection. Set by the consumer at WO creation. New kinds are added without DDL because the kind is configured in catalog rows.
Job type code	Fine-grained code from the per-operator catalog (wo_job_type_catalog). Each operator populates its own codes. Drives Drools rules, note-kind requirements, picklist filtering.
Phase	A sub-state within status=IN_PROGRESS. Multi-phase WO flows use phases (e.g., DISCONNECT / RECONNECT) to split work within one process instance.
Single-WO multi-phase	The model where one wo_id carries an entire physical operation. The job_type_code may transition between phases. Used by SHIFTING; configurable per kind.
Caller context	The subscription-workflow-supplied metadata stored on the WO (subscriptionId, operationId, callerWorkflowKind, callerWorkflowRefId). The WO module persists it in wo_metadata and emits it on every Kafka event for caller correlation.

1. The integration model

1.1 One WO per physical operation

A physical operation that touches the customer premises generates **one Work Order**, not multiple. For multi-step operations (e.g., a relocation involving both source-side disconnect and target-side install work), the WO module uses its multi-phase mechanism: one wo_id cycles through phases (e.g., DISCONNECT → RECONNECT) within a single Camunda process instance.

```
WO created (status=PENDING, current_phase=NULL)
↓
WO assigned, started (status=IN_PROGRESS, current_phase=<first phase>, job_type_code=<first phase code>)
↓
First-phase work completed → first-confirm finalize → second-confirm finalize
↓
[multi-phase only] PHASE TRANSITION (current_phase advances; job_type_code may swap)
↓
```

```

[multi-phase only] Next-phase work begins (still status=IN_PROGRESS)
↓
[multi-phase only] Next-phase work completed → first-confirm finalize → second-confirm finalize
↓
WO COMPLETED (status=COMPLETED)

```

Single-phase WOs (e.g., a fresh installation) finalize once. Multi-phase WOs finalize per phase + final WO completion.

1.2 Kafka events subscription workflows park on

The WO module emits framework events (defined in DD_WO-01-FRAMEWORK §7) at every state transition. Subscription workflows wire Camunda intermediate message catch events to these topics, correlated on `callerContext.operationId`:

Event	Topic	Fires when	Typical use
WorkOrderCreated	sophix.work-orders.created	WO row created, process instance started	Optional — workflow already knows it just created the WO
WorkOrderPhaseTransitioned	sophix.work-orders.phase-transitioned	current_phase advances within a multi-phase WO	Workflow learns intermediate work is done; may run interleaved steps (e.g., FUL-04 SUSPEND_SERVICE between source-side and target-side work)
WorkOrderPhaseFinalized	sophix.work-orders.phase-finalized	A phase's second-confirm finalize completes	Alternative to WorkOrderPhaseTransitioned if workflow needs to act between phase-finalize and next-phase-start
WorkOrderFinalized	sophix.work-orders.finalized	Entire WO's second-confirm finalize completes	Workflow knows all physical work is done; proceeds to FUL-03 / FUL-07 / commit
WorkOrderCancelled	sophix.work-orders.cancelled	Dispatcher (or caller via cancel API) cancels the WO	Workflow runs its own compensation chain
WorkOrderTimedOut	sophix.work-orders.timed-out	WO exceeds wo_kind_config.default_timeout_days	Workflow treats as failure; reverts; emits its own Rejected event

Subscription workflow BPMN registers message catches correlated on `callerContext.operationId` (which the workflow supplied at WO creation). The WO module includes `callerContext` in every event payload — the workflow's catches resolve to the right process instance.

1.3 WO creation endpoint

POST /api/work-orders — per DD_WO-01-FRAMEWORK §6. Single endpoint for all kinds.

```

POST /api/work-orders HTTP/1.1
Content-Type: application/json

```

```

{
  "operatorCode":      "<operator>",
  "kind":              "<KIND>",                // INSTALLATION / SHIFTING / TECHNOLOGY_MIGRATION / operator extension
  "jobTypeCode":       "<code>",                // per operator's wo_job_type_catalog
  "subscriptionId":    "sub_...",
  "customerId":        "cust_...",
  "homepassRefs": {
    "primary":         "hp_...",                // e.g., new address for SHIFTING; install address for INSTALLATION
    "secondary":       "hp_..."              // e.g., old address for SHIFTING; null for single-location ops
  },
  "currentEquipmentBindings": [
    { "equipmentId": "...", "serviceClassId": "..." }
  ],
  "scheduledAt":       "2026-06-01T09:00:00+03:00",
  "flowContext": { ... },                        // flow-specific fields (e.g., equipmentPath for SHIFTING)
  "callerContext": {
    "subscriptionId":   "sub_...",
    "operationId":      "subop_...",
    "callerWorkflowKind": "SUB_WF_RELOCATION",    // SUB_WF_ACTIVATE / SUB_WF_RELOCATION / SUB_WF_MIGRATION / etc.
    "callerWorkflowRefId": "subop_..."
  }
}

→ 201 Created
{
  "id":                "wo_...",
  "status":            "PENDING",
  "currentPhase":      null,
  "jobTypeCode":       "<code>",
  "kind":              "<KIND>"
}

```

```
}
```

homepassRefs.primary and homepassRefs.secondary cover one- or two-location physical operations. For an INSTALLATION, only primary is set (install address). For a SHIFTING, both are set (primary = new, secondary = old). For TECHNOLOGY_MIGRATION, use the same shape when both old and new HomePass references are relevant.

1.4 Job type code selection

Job type codes are operator-scoped. Each operator's wo_job_type_catalog defines its own codes for each kind. Subscription workflows resolve the code via Drools at WO-creation time, using the source HomePass's network root type and the target Package's service classes as input.

The selection rule (operator-neutral form):

```
sourceNetworkRootType = sourceHomePass.networkPath.rootNodeType // e.g., a fiber root type, a coax root type
flowContext.equipmentPath in {KEEP_EXISTING, NEW_INSTALL}
flowContext.crossTech = (sourceNetworkRootType != targetNetworkRootType)
```

For each operator, Drools maps (kind, sourceNetworkRootType, equipmentPath, crossTech) → jobTypeCode. The mapping is operator-specific — operators register their codes in wo_job_type_catalog and their Drools KJAR. Each operator's catalog populates its own codes; no canonical cross-operator codeset is assumed by this contract.

For multi-phase WOs that swap codes mid-flow (e.g., disconnect → reconnect), the WO module handles the transition internally; the subscription workflow only specifies the FIRST phase's code at creation.

1.5 Equipment handling

The WO module handles equipment binding internally. Subscription workflows do not call separate osr-equipment-bind / osr-equipment-unbind workers for operations that have a WO.

At WO creation, the workflow passes currentEquipmentBindings and flowContext.equipmentPath. The WO's internal logic (FLOW-specific) decides whether to:

- Record new equipment serials captured by the tech at finalize-time (NEW_INSTALL)
- Carry forward existing equipment between locations (KEEP_EXISTING with transport)
- Recover and return existing equipment to inventory (NEW_INSTALL with old-equipment recovery)

The WorkOrderFinalized event payload includes finalEquipmentBindings[] — the post-WO equipment state. Subscription workflows read this for downstream FUL-03 ACTIVATE_SERVICE calls + FUL-07 CLAIM context.

For operations where the customer self-handles equipment (no tech dispatch — e.g., disconnectMode = CUSTOMER_SELF_MOVE in subscription terms), the subscription workflow may skip WO creation entirely. The integration choice is the workflow's: WO = tech dispatched; no WO = customer self-service.

1.6 HomePass-side bookkeeping responsibilities

Two modules touch HomePass state — and they coexist with clear ownership:

Module	Owns at HomePass	Triggered by
WO-01 (FLOW-specific)	Account sub-status field on the customer's HomePass-anchored account; HomePass topology fields confirmed or captured at finalize; equipment bindings on the WO	Inside the WO's BPMN at phase-finalize and WO-finalize steps
FUL-07	homepass_subscription_binding rows (one per active subscription per HomePass); active_termination_points counter on the HomePass; per-port bindings for AT_INSTALLATION HomePasses	Subscription workflow worker calls (ful-call-release-homepass, ful-call-claim-homepass) at workflow steps independent of WO progress

Both run on the same physical operation but at different lifecycle moments. The subscription workflow orchestrates them in its own sequence. WO-01 does its work autonomously inside the WO process instance.

The account sub-status codes WO-01 sets are per-operator (each operator has its own account-sub-status catalog). The WO event payload field accountSubStatusChange = {to: "<operator code>", reason: "<flow-specific reason>"} is consumed by the operator's ILM module.

1.7 Timeouts and cancellation

WO timeouts come from the WO module's `wo_kind_config.default_timeout_days` table (per DD_WO-01-FRAMEWORK §7). Subscription workflows reference this single source — they don't carry their own duplicate timeout columns.

`WorkOrderTimedOut` event fires when the timeout is exceeded. The subscription workflow's compensation path runs on receipt: revert state to pre-workflow snapshot, emit `Rejected` event with `rejectionReasonCode = WO_TIMEOUT`, finalize `FAILED`.

Cancellation flows in two directions:

- **Subscription workflow** → **WO**: if customer cancels the workflow request (before commit window), the workflow calls `POST /api/work-orders/{id}/cancel` to abort the WO. WO-01 terminates the process instance and emits `WorkOrderCancelled`.
- **WO** → **subscription workflow**: if dispatcher cancels the WO (operations intervention), `WorkOrderCancelled` arrives at the workflow's catch. Workflow runs compensation.

If the workflow is **in the commit window** (atomic-switch in progress, past the point of no return), the cancel REST endpoint on the subscription side returns `409 OPERATION_IN_COMMIT_WINDOW`. Whether the WO can still be cancelled at that point is the WO module's decision — operations may force-cancel a WO regardless of subscription-side state, in which case the subscription workflow handles the unexpected cancel via its existing failure-mode compensation.

1.8 Cross-tech and complex variations

Subscription operations that involve **cross-technology HomePass moves** (for example source on legacy network root type, target on current network root type) are coordinated by `FUL-10 Technology Migration Fulfillment`. When a field visit is needed, FUL-10 creates one or more WOs through this same endpoint using either:

- existing `SHIFTING` with operator-specific job-type codes when the operator's field process is close to shifting; or
- `TECHNOLOGY_MIGRATION` when the operator needs a dedicated cross-tech field workflow.

Both are supported by the framework — `kind` is a config-driven enum, not a hardcoded one. Operators register their own kinds as their needs evolve, and subscription workflows pick the kind via the same Drools-resolved mapping in §1.4.

1.9 Operator-specific FLOW behavior is the FLOW satellite's concern

Anything that varies per operator — account sub-status codes, equipment recovery policy at disconnect, topology-capture vs topology-confirm at reconnect, multi-phase vs single-phase structure for a given operation type, contractor-payment side-effects, finalize-checklist note kinds, capture mechanism (back-office vs field-app) — lives in the per-operator FLOW satellite (DD_WO-01-FLOW-<KIND>-<OPERATOR>) or in operator-scoped catalogs and KJAR packages. Subscription workflows are insulated from those choices by the integration surface defined here: REST, Kafka, caller context, framework events.

2. Integration surface summary

2.1 Endpoints subscription workflows call

Use	Endpoint
Create a WO for a subscription operation	<code>POST /api/work-orders</code>
Cancel a pending or in-progress WO from the workflow side	<code>POST /api/work-orders/{id}/cancel</code>

2.2 Topics subscription workflows subscribe to

Topic	When the workflow cares
<code>sophix.work-orders.phase-transitioned</code>	Multi-phase WOs — between-phase action point
<code>sophix.work-orders.phase-finalized</code>	Multi-phase WOs — alternative between-phase trigger
<code>sophix.work-orders.finalized</code>	Single-phase OR end-of-multi-phase — WO entirely done
<code>sophix.work-orders.cancelled</code>	WO cancelled by dispatcher or by the workflow itself
<code>sophix.work-orders.timed-out</code>	WO exceeded operator-configured timeout

2.3 Correlation contract

Subscription workflow BPMN catches correlate on `callerContext.operationId`. The WO module includes `callerContext` in every event payload it emits. The workflow's BPMN doesn't need to track the `wo_id` — correlation by operation ID is sufficient.

If a single subscription operation creates multiple WOs (unusual, but possible — e.g., a complex migration that spans multiple geographies), the workflow uses additional sub-keys on `callerContext` (e.g., `callerContext.legId`) to disambiguate.

2.4 Caller context fields

Field	Required	Purpose
<code>subscriptionId</code>	yes	Identifies the subscription; useful for operations audit
<code>operationId</code>	yes	Camunda message correlation key
<code>callerWorkflowKind</code>	yes	Coarse classification (SUB_WF_ACTIVATE / SUB_WF_RELOCATION / SUB_WF_MIGRATION / SUB_WF_TERMINATE / ...) — used by reporting + Kafka filtering
<code>callerWorkflowRefId</code>	yes	Specific workflow operation reference (typically same as <code>operationId</code>)
<code>legId</code>	no	Sub-key for multi-WO single operations

2.5 Flow context fields

`flowContext` is opaque to the integration surface and interpreted by the FLOW satellite. Subscription workflows or FUL-10 populate it per the FLOW's requirements. Common shape for HomePass-change flows (SHIFTING and TECHNOLOGY_MIGRATION):

```
{
  "equipmentPath": "KEEP_EXISTING | NEW_INSTALL",
  "disconnectMode": "TECH_DISPATCH | CUSTOMER_SELF_MOVE",
  "sourceHomepassId": "hp_...", // also in homepassRefs.secondary
  "targetHomepassId": "hp_..." // also in homepassRefs.primary
}
```

Single-location flows (INSTALLATION) populate fewer fields. Each FLOW satellite specifies its own.

3. Cross-DD coordination

Module	This DD's relationship
DD_WO-01-FRAMEWORK-v1.0	Authoritative source for WO state machine, REST API, Kafka events, schemas, <code>wo_kind_config</code> , <code>wo_job_type_catalog</code> . This DD references the framework primitives.
DD_WO-01-FLOW-INSTALLATION-v1.0	One concrete instance of the integration surface — single-phase WO consumed by SUB-WF-ACTIVATE-01.
DD_WO-01-FLOW-SHIFTING-v1.0	One concrete instance of the integration surface — multi-phase WO consumed by SUB-WF-RELOCATION-01 (same-tech case).
FUL-10 Technology Migration Fulfillment	Owns the technical cross-technology migration sequence. Calls WO-01 only for field work that needs dispatch/execution.
DD_WO-01-FLOW-TECHNOLOGY-MIGRATION-*	Optional concrete WO flow for operators whose cross-technology migration cannot reuse SHIFTING.
SUB-WF-ACTIVATE-01	Consumer of <code>kind=INSTALLATION</code> WOs.
SUB-WF-RELOCATION-01	Consumer of <code>kind=SHIFTING</code> WOs for same-technology moves.
SUB-WF-MIGRATION-01	Consumer of <code>kind=SHIFTING</code> WOs for same-technology cases; delegates cross-technology technical fulfillment to FUL-10.
SUB-WF-TERMINATE-01	Does not create WOs synchronously. Hands off equipment recovery to OSR-RMA-01 via <code>SubscriptionTerminated</code> event consumer; OSR-RMA-01 may create its own pickup WO downstream.

4. Build order

1. DD_WO-01-FRAMEWORK-v1.0 (already shipped)

2. DD_WO-01-FLOW-INSTALLATION-v1.0 (already shipped)
3. DD_WO-01-FLOW-SHIFTING-v1.0 (already shipped)
4. This contract DD (informational, references existing framework + flows)
5. Subscription-workflow DDs that consume the contract
6. Future per-operator FLOW satellites added without DDL as operators' deep-dives surface their variations

B — Current TZ

Not applicable. This DD is an integration-contract specification document. The integration surface it specifies references framework + flow DDs whose own §B sections carry their TZ baseline placeholders.